

FULL STACK DEVELOPMENT

FASE 4 | AULA 01 - INTRODUÇÃO AO DESENVOLVIMENTO MOBILE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	21
REFERÊNCIAS.....	22

EMSE

O QUE VEM POR AÍ?

Nesta aula, você terá um contexto de como funciona o ambiente de desenvolvimento mobile de forma geral, as plataformas disponíveis e formas de desenvolver para plataformas específicas ou para multiplataformas. Também veremos vantagens e desvantagens das tecnologias mais utilizadas.

Além disso, também abordaremos como efetuar a configuração do ambiente de desenvolvimento para que possamos desenvolver e executar nosso projeto tranquilamente.

EXEMPLO

HANDS ON

Nesta aula, abordaremos todo o ecossistema do desenvolvimento mobile, aprenderemos sobre diversas tecnologias, configuraremos nosso ambiente de desenvolvimento e criaremos parte do esqueleto do nosso projeto. Vamos lá?



SAIBA MAIS

Segue código trabalhado durante o nosso Hands On:

```
Nvm install 20.12.1
```

```
Npm i -g expo-cli
```

```
Npx create-expo-app Clima-Tempo
```

```
import React, {useState} from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, TextInput, Alert, Button } from 'react-native';

export default function App() {
  const [user, setUser] = useState("");
  const [password, setPassword] = useState("");

  const handleInputChange = (inputText) => {
    setText(inputText);
  };

  const handleButtonPress = () => {
    Alert.alert('Usuario digitado:', user);
    Alert.alert('Senha digitada:', password);
  };

  return (
    <View style={styles.container}>
      <Text>Clima Tempo</Text>
      <Text>Digite seu usuário</Text>
    </View>
  );
}
```

```
<TextInput style={{ height: 40, borderColor: 'gray', borderWidth: 1, marginBottom:
10, paddingHorizontal: 10 }}

  onChangeText={(inputText)=>{setUser(inputText)}}

  value={user}

/>

<Text>Digite sua senha</Text>

<TextInput/>

<TextInput style={{ height: 40, borderColor: 'gray', borderWidth: 1, marginBottom:
10, paddingHorizontal: 10 }}

  onChangeText={(inputText)=>{setPassword(inputText)}}

  value={password}

/>

<StatusBar style="auto" />

<Button title='Enviar' onPress={handleButtonPress}/>

</View>

);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Npx expo start

O que é o desenvolvimento mobile

O desenvolvimento mobile refere-se ao processo de criação de aplicativos e softwares específicos para dispositivos móveis, como smartphones e tablets. Esses aplicativos são desenvolvidos para serem executados em sistemas operacionais móveis, como iOS (para dispositivos Apple, como iPhone e iPad) e Android (para uma variedade de dispositivos de diferentes fabricantes).

A principal característica distintiva do desenvolvimento móvel é a adaptação do software às restrições e recursos específicos dos dispositivos móveis. Isso inclui considerações como tamanho de tela variável, interações baseadas em toque, conectividade de rede intermitente e consumo eficiente de energia.

A definição de desenvolvimento móvel também engloba a criação de aplicativos que tiram proveito dos recursos exclusivos dos dispositivos móveis, como câmeras, sensores de localização, acelerômetros e tecnologias de comunicação sem fio (como Bluetooth e NFC). Esses recursos permitem que os aplicativos móveis forneçam experiências interativas e personalizadas, adaptadas ao estilo de vida e ao ambiente em constante mudança dos usuários móveis.

Crescimento do mercado de aplicações mobile

A importância e o crescimento do mercado de aplicativos móveis são fenômenos significativos impulsionados por uma série de fatores que transformaram fundamentalmente a maneira como as pessoas interagem com a tecnologia e com o mundo ao seu redor. Aqui estão algumas razões que destacam a importância e o crescimento contínuo desse mercado:

Acesso ubíquo à tecnologia: os smartphones se tornaram uma parte indispensável da vida cotidiana para bilhões de pessoas em todo o mundo. Eles oferecem acesso instantâneo à internet, comunicação, entretenimento, informações e uma variedade de serviços por meio de aplicativos móveis.

Expansão da conectividade e tecnologia móvel: com o avanço das redes de telefonia móvel, como o 4G e 5G, e o aumento da inserção de smartphones em

mercados emergentes, mais pessoas têm acesso à internet em seus dispositivos móveis do que nunca. Isso abre um vasto mercado para aplicativos móveis em todo o mundo.

Economia de aplicativos: o mercado de aplicativos móveis tornou-se uma economia próspera por si só. Milhões de aplicativos estão disponíveis em lojas de aplicativos, gerando receita por meio de modelos de negócios como publicidade, compras dentro do aplicativo, assinaturas e downloads pagos.

Facilidade de distribuição e monetização: as lojas de aplicativos, como a App Store da Apple e o Google Play, oferecem às pessoas desenvolvedoras uma plataforma centralizada para distribuir e monetizar seus aplicativos. Isso simplifica o processo de alcançar uma audiência global e permite que desenvolvedores independentes compitam no mercado.

Diversidade de aplicativos e serviços: os aplicativos móveis abrangem uma ampla gama de categorias, incluindo redes sociais, jogos, utilitários, saúde, finanças, educação, produtividade e muito mais. Isso cria oportunidades para uma variedade de desenvolvedores(as) e empresas de todos os tamanhos, atendendo a uma ampla gama de necessidades e interesses dos usuários.

Transformação digital empresarial: as empresas estão cada vez mais reconhecendo a importância de ter uma presença móvel para alcançar e engajar clientes. Desde pequenas empresas até grandes corporações, investir em aplicativos móveis tornou-se uma parte crucial das estratégias de transformação digital para permanecerem competitivas no mercado atual.

Experiência do usuário aprimorada: os aplicativos móveis oferecem uma experiência de usuário otimizada e personalizada, adaptada aos dispositivos móveis e às preferências individuais dos usuários. Isso permite interações mais intuitivas e eficientes, resultando em maior satisfação do usuário e fidelidade à marca.

Inovação contínua: o mercado de aplicativos móveis está em constante evolução, impulsionado pela inovação tecnológica e pelas demandas dos usuários. Novas tecnologias, como realidade aumentada, inteligência artificial e Internet das Coisas, estão sendo incorporadas aos aplicativos móveis, criando novas oportunidades e experiências únicas para os usuários.

Ambientes mobile e suas características

No desenvolvimento móvel, as duas principais plataformas são iOS, desenvolvida pela Apple, e Android, desenvolvida pelo Google. Ambas dominam o mercado global de smartphones, cada uma com suas próprias características, ferramentas e ecossistemas de desenvolvimento. Veja uma visão geral das principais plataformas:

iOS (Apple)

Desenvolvido pela Apple, o iOS é o sistema operacional exclusivo para dispositivos como iPhone, iPad e iPod Touch. A linguagem de programação principal para o desenvolvimento iOS é o Swift, embora Objective-C ainda seja suportado.

O ambiente de desenvolvimento integrado (IDE) oficial da Apple é o Xcode, que oferece uma variedade de ferramentas para desenvolvimento, teste e depuração de aplicativos iOS.

A App Store é a plataforma oficial de distribuição de aplicativos para dispositivos iOS, e os(as) desenvolvedores(as) devem seguir as diretrizes rigorosas da App Store para que seus aplicativos sejam aprovados e listados na loja.

A Apple é conhecida por seu ecossistema fechado, o que significa que ela mantém um alto controle sobre o hardware e o software de seus dispositivos, proporcionando uma experiência coesa e consistente para os usuários.

Android (Google)

Desenvolvido pelo Google, o Android é o sistema operacional mais utilizado em dispositivos móveis em todo o mundo, abrangendo uma ampla gama de fabricantes e dispositivos. A linguagem de programação principal para o desenvolvimento Android é Java, embora Kotlin tenha se tornado uma opção popular e oficialmente suportada pelo Google.

O Android Studio é o IDE oficial do Google para desenvolvimento Android, oferecendo uma variedade de recursos e ferramentas para facilitar o processo de desenvolvimento.

O Google Play Store é a plataforma oficial de distribuição de aplicativos para dispositivos Android, oferecendo aos desenvolvedores uma ampla audiência global.

O Android é conhecido por seu ecossistema mais aberto, que oferece aos(as) desenvolvedores(as) maior flexibilidade e liberdade para personalizar e modificar o sistema operacional e os aplicativos.

Ainda podemos mencionar: wearOs para dispositivos wearable, como smartwatches, e iPadOS para tablets iPad.

Swift

Swift é uma linguagem de programação desenvolvida pela Apple, lançada em 2014 como uma alternativa ao Objective-C para o desenvolvimento de aplicativos para iOS, macOS, watchOS e tvOS. Ela foi introduzida com o objetivo de oferecer uma linguagem moderna, poderosa e mais fácil de aprender e usar do que o Objective-C.

Essa linguagem foi criada pela equipe de engenharia da Apple, liderada por Chris Lattner, como uma resposta às limitações e complexidades percebidas do Objective-C. A Swift foi projetada para ser mais segura, rápida e eficiente, incorporando conceitos modernos de programação e características inspiradas em outras linguagens populares.

Vantagens

- **Segurança avançada:** foi projetada para ser uma linguagem segura desde o início, ajuda as pessoas desenvolvedoras a evitar erros comuns de programação, como nil pointer dereferencing, tornando os aplicativos mais seguros e menos propensos a falhas.
- **Sintaxe concisa e expressiva:** a sintaxe de Swift é moderna, clara e expressiva, o que torna a escrita de código mais eficiente e legível. Sua sintaxe simplificada reduz a quantidade de código necessária para realizar tarefas comuns, o que pode aumentar a produtividade dos desenvolvedores.
- **Desempenho aprimorado:** altamente otimizada para o desempenho, oferecendo tempos de execução mais rápidos e menor consumo de memória em comparação com o Objective-C. Isso resulta em aplicativos mais rápidos, responsivos e eficientes em termos de recursos.

- Interoperabilidade com Objective-C: Swift foi projetada para ser totalmente interoperável com o Objective-C, permitindo que os(as) desenvolvedores(as) usem ambas as linguagens em um mesmo projeto. Isso facilita a migração gradual de código legado para Swift e aproveita o vasto ecossistema de bibliotecas e frameworks existentes em Objective-C.
- Fácil manutenção: a sintaxe clara e expressiva de Swift, combinada com recursos como inferência de tipos e generics, torna o código mais fácil de entender e manter. Isso pode reduzir o tempo e os esforços necessários para realizar tarefas de manutenção e refatoração no código.

Desvantagens

- Curva de aprendizado inicial: embora Swift tenha sido projetada para ser mais acessível e fácil de aprender do que o Objective-C, ainda pode representar uma curva de aprendizado para os(as) desenvolvedores(as) que estão migrando de outras linguagens de programação. Além disso, a linguagem continua a evoluir com o tempo, o que pode exigir que quem desenvolve tenha que acompanhar as mudanças e atualizações.
- Compatibilidade com versões anteriores: a Apple continua a introduzir novas versões do Swift com recursos e melhorias adicionais. Isso pode resultar em problemas de compatibilidade entre diferentes versões do Swift, especialmente ao atualizar projetos existentes para versões mais recentes da linguagem.
- Evolução do ecossistema: como uma linguagem relativamente nova, o ecossistema de Swift ainda está em evolução. Algumas bibliotecas e ferramentas de terceiros podem não oferecer suporte total ou otimizado para ela, o que pode limitar as opções disponíveis para os desenvolvedores.
- Ferramentas de depuração e desenvolvimento: embora o Xcode ofereça um conjunto robusto de ferramentas para desenvolvimento e depuração de aplicativos Swift, algumas ferramentas de terceiros podem não oferecer suporte completo ou otimizado. Isso pode afetar a experiência de quem desenvolve ao trabalhar com a linguagem.

Apesar de suas desvantagens, Swift continua sendo uma escolha popular e poderosa para o desenvolvimento de aplicativos iOS devido às suas vantagens significativas em termos de segurança, desempenho e usabilidade. Sua evolução contínua e o apoio da comunidade de desenvolvimento e da Apple indicam um futuro promissor para a linguagem Swift no ecossistema.

Java para android

Vantagens

- **Maturidade e estabilidade:** Java é uma linguagem de programação estabelecida e amplamente utilizada, com uma vasta base de conhecimento e suporte da comunidade.
- **Ecossistema rico:** existe um grande número de bibliotecas, frameworks e ferramentas disponíveis para desenvolvimento Android em Java, proporcionando a quem desenvolve uma ampla gama de recursos para criar aplicativos.
- **Compatibilidade com versões anteriores:** como uma linguagem mais antiga, Java oferece uma boa compatibilidade com versões mais antigas do Android, o que pode ser importante para aplicativos que precisam atingir uma base de usuários mais ampla.
- **Integração com outras tecnologias:** ele é amplamente utilizado em outras áreas além do desenvolvimento móvel, o que pode facilitar a integração de aplicativos móveis com sistemas e serviços corporativos.

Desvantagens

- **Código verboso:** Java tende a ser mais verboso em comparação com linguagens mais modernas, o que pode resultar em mais código e reduzir a legibilidade.
- **Gerenciamento manual de nulos:** ele não possui um sistema de tipos que lida automaticamente com valores nulos, o que pode aumentar a probabilidade de erros relacionados a nulos.

- Menos funcionalidades modernas: Java pode carecer de algumas das características mais avançadas encontradas em linguagens mais recentes, como inferência de tipos, extensões de funções e expressões lambda.

Kotlin

Vantagens

- Conciso e expressivo: Kotlin possui uma sintaxe mais concisa e expressiva em comparação com Java, o que pode reduzir a quantidade de código necessário e melhorar a legibilidade.
- Segurança aprimorada: oferece recursos de segurança, como suporte nativo para nulos e tipos não nulos, que podem ajudar a prevenir erros comuns de programação.
- Integração perfeita com Java: Kotlin é totalmente interoperável com Java, o que significa que os(as) desenvolvedores(as) podem usar as duas linguagens em um mesmo projeto e aproveitar o vasto ecossistema de bibliotecas e frameworks existentes.
- Recursos modernos: oferece uma série de recursos modernos de linguagem, como inferência de tipos, extensões de funções, expressões lambda e corrotinas, que podem melhorar a produtividade do desenvolvedor e a qualidade do código.

Desvantagens

- Curva de aprendizado: embora Kotlin seja projetado para ser mais acessível e fácil de aprender do que Java, ainda pode representar uma curva de aprendizado para as pessoas desenvolvedoras que estão migrando de outras linguagens.
- Maturidade relativa: é uma linguagem relativamente nova em comparação com Java, o que pode resultar em menos recursos e suporte da comunidade em comparação com Java.

- Tamanho do binário: em alguns casos, o tamanho do binário gerado por Kotlin pode ser ligeiramente maior do que o de aplicativos Java, o que pode afetar o desempenho e o consumo de recursos do aplicativo.

Em resumo, tanto Java quanto Kotlin são opções viáveis para o desenvolvimento de aplicativos móveis Android, cada um com suas próprias vantagens e desvantagens. A escolha entre essas duas linguagens depende das necessidades específicas do projeto, preferências pessoais e familiaridade com a linguagem. No entanto, Kotlin tem ganhado popularidade significativa nos últimos anos devido às suas características modernas e suporte nativo do Google.

Ciclos de vida do desenvolvimento de aplicações mobile

1. Concepção e planejamento

Nesta fase inicial, a equipe define o propósito e os objetivos do aplicativo. São identificados os requisitos do usuário, as funcionalidades principais e a estratégia de lançamento. Podem ser realizadas pesquisas de mercado e análises de concorrência para informar o planejamento.

2. Design de Interface do Usuário (UI/UX)

O design da interface do usuário é crucial para a usabilidade e a experiência do usuário. São criados wireframes, protótipos e designs visuais para representar a aparência e o comportamento do aplicativo.

O objetivo é criar uma experiência intuitiva e atrativa para os usuários.

3. Desenvolvimento

Nesta fase, o aplicativo é construído com base nos requisitos e designs estabelecidos. As pessoas desenvolvedoras escrevem o código-fonte, implementam as funcionalidades e integram APIs externas, quando necessário. Testes unitários são realizados para garantir a qualidade do código e a funcionalidade das partes individuais do aplicativo.

4. Testes e depuração

O aplicativo passa por uma série de testes para identificar e corrigir bugs e problemas de desempenho, que podem incluir testes de unidade, testes de integração, testes de aceitação do usuário e testes de desempenho.

A depuração é um processo contínuo para garantir que o aplicativo funcione conforme o esperado em uma variedade de dispositivos e condições.

5. Implantação e distribuição

Após os testes bem-sucedidos, o aplicativo é preparado para lançamento. Ele é submetido às lojas de aplicativos relevantes, como a App Store da Apple e o Google Play Store para dispositivos Android. São fornecidos metadados, como descrição do aplicativo, imagens de tela e informações de classificação, conforme as diretrizes da loja de aplicativos.

6. Lançamento e avaliação

O aplicativo é lançado publicamente para os usuários baixarem e utilizarem. Os(as) desenvolvedores(as) monitoram a recepção do aplicativo, coletam feedback dos usuários e respondem a quaisquer problemas ou solicitações.

É importante iterar e lançar atualizações frequentes para melhorar a experiência do usuário e corrigir problemas identificados.

7. Manutenção e atualização

Após o lançamento inicial, o aplicativo requer manutenção contínua para garantir que ele permaneça funcional e compatível com as atualizações do sistema operacional e de hardware. Atualizações periódicas são lançadas para adicionar novas funcionalidades, corrigir bugs e manter a relevância do aplicativo.

Estrutura do React Native

A estrutura do React Native é projetada para fornecer uma maneira eficaz de criar aplicativos móveis nativos para iOS e Android usando JavaScript e React. Aqui está uma análise detalhada da estrutura, incluindo suas vantagens, desvantagens, casos de uso e visão de escalabilidade:

JavaScript e React: o React Native utiliza JavaScript como a linguagem de programação principal, juntamente com React, uma biblioteca JavaScript para

construção de interfaces de usuário. Isso permite que os(as) desenvolvedores(as) usem as mesmas habilidades e ferramentas que já conhecem para criar aplicativos móveis nativos.

Componentes reutilizáveis: a estrutura do React Native é baseada no conceito de componentes reutilizáveis. As pessoas desenvolvedoras podem criar componentes que representam partes específicas da interface do usuário, como botões, campos de texto, listas e muito mais. Esses componentes podem ser reutilizados em todo o aplicativo para criar uma interface consistente e coesa.

Acesso às APIs nativas: uma das vantagens do React Native é sua capacidade de acessar e interagir com APIs nativas de cada plataforma (iOS e Android) diretamente do JavaScript. Isso é possível por meio de uma camada de abstração que expõe as APIs nativas como módulos JavaScript acessíveis no código do aplicativo. Isso permite que quem desenvolve crie aplicativos com funcionalidades nativas sem sacrificar a portabilidade do código.

Hot Reloading e Live Reload: React Native suporta recursos de desenvolvimento como Hot Reloading e Live Reload, que permitem aos(as) desenvolvedores(as) visualizar instantaneamente as mudanças feitas no código sem a necessidade de reiniciar o aplicativo. Isso acelera significativamente o processo de desenvolvimento e teste.

Estilos Flexbox: o layout no React Native é gerenciado por meio de um sistema de estilo baseado em Flexbox, que permite criar layouts responsivos e flexíveis que se adaptam a diferentes tamanhos de tela e orientações.

Bibliotecas e módulos nativos: além das APIs nativas incorporadas, os(as) profissionais de desenvolvimento podem usar bibliotecas e módulos nativos de terceiros para adicionar funcionalidades adicionais aos seus aplicativos. Existem milhares de bibliotecas disponíveis através do npm (Node Package Manager) que podem ser instaladas e usadas em projetos React Native.

Portabilidade do código: uma das principais vantagens do React Native é sua capacidade de compartilhar código entre iOS e Android. Embora algumas partes do aplicativo possam precisar ser específicas de plataforma, a maior parte da lógica de negócios e da interface do usuário pode ser compartilhada entre as plataformas.

Vantagens

- **Desenvolvimento rápido:** React Native permite um desenvolvimento mais rápido de aplicativos móveis, graças à sua sintaxe simplificada e à capacidade de reutilizar componentes.
- **Acesso a APIs nativas:** os(as) desenvolvedores(as) podem acessar facilmente funcionalidades nativas de dispositivos usando módulos JavaScript, proporcionando uma experiência de aplicativo mais rica e integrada.
- **Portabilidade do código:** a capacidade de compartilhar a maior parte do código entre iOS e Android economiza tempo e esforço, tornando o desenvolvimento mais eficiente.
- **Comunidade ativa:** React Native possui uma comunidade ativa de desenvolvedores(as) e uma grande quantidade de bibliotecas e recursos disponíveis para uso.

Desvantagens

- **Desempenho:** embora React Native ofereça um bom desempenho em muitos casos, aplicativos mais complexos podem experimentar alguma degradação de desempenho devido à camada de abstração entre JavaScript e APIs nativas.
- **Compatibilidade com novas funcionalidades:** algumas funcionalidades nativas podem não ser suportadas imediatamente no React Native ou podem exigir o uso de bibliotecas de terceiros.
- **Dependência de atualizações:** a dependência de atualizações do React Native e de suas dependências pode causar problemas de compatibilidade e quebra de funcionalidades em algumas situações.

Casos de uso

Aplicativos empresariais e corporativos, de comércio eletrônico, de mídia social, de entrega de conteúdo, de serviços de streaming, de saúde e bem-estar, de utilidades e ferramentas, entre outros.

Casos de uso reais

Facebook: utiliza React Native em vários de seus produtos, incluindo o aplicativo Facebook, Messenger e o Instagram. Isso demonstra a capacidade do React Native de lidar com aplicativos de grande escala e alta demanda de usuários.

Walmart: a gigante do varejo adotou o React Native para desenvolver seu aplicativo móvel, que oferece uma experiência de compra conveniente e integrada para os clientes, incluindo recursos como busca de produtos, navegação, compras online e retirada em loja.

Airbnb: a plataforma de aluguel de hospedagens utiliza o React Native para seu aplicativo móvel, permitindo aos usuários pesquisar e reservar acomodações em todo o mundo de forma rápida e intuitiva. O uso do React Native permitiu à equipe de desenvolvimento alcançar uma base de usuários diversificada em diferentes dispositivos e sistemas operacionais.

Visão de escalabilidade

React Native oferece uma boa escalabilidade para aplicativos móveis, permitindo que equipes de desenvolvimento construam e expandam aplicativos de todos os tamanhos e complexidades. No entanto, à medida que os aplicativos crescem, é importante considerar estratégias de otimização de desempenho e gerenciamento de estado para garantir que o aplicativo permaneça rápido, responsivo e fácil de manter. A modularização do código, a implementação de padrões de arquitetura sólidos e a adoção de práticas de teste robustas são essenciais para garantir uma escalabilidade eficaz do aplicativo React Native. Além disso, a participação ativa na comunidade de desenvolvimento pode fornecer suporte e orientação valiosos ao enfrentar desafios de escalabilidade.

Ecossistema React Native

O ecossistema do React Native é rico e diversificado, com uma ampla gama de ferramentas, bibliotecas, frameworks e recursos disponíveis para quem desenvolve. Aqui estão alguns tópicos importantes dentro do ecossistema do React Native:

React Native CLI vs. Expo: as pessoas desenvolvedoras têm a opção de usar o React Native CLI ou o Expo para iniciar e gerenciar projetos React Native. O React Native CLI oferece mais controle sobre o projeto e acesso direto às APIs nativas, enquanto o Expo simplifica o desenvolvimento, fornecendo um conjunto de ferramentas e serviços integrados.

Ferramentas de desenvolvimento: o ecossistema do React Native oferece uma variedade de ferramentas de desenvolvimento, como editores de código (VSCode, Atom), emuladores (Android Studio, Xcode), depuradores e extensões que tornam o processo de desenvolvimento mais eficiente e produtivo.

Bibliotecas e Frameworks: existem milhares de bibliotecas e frameworks disponíveis para React Native que ajudam os(as) desenvolvedores(as) a adicionar funcionalidades específicas aos seus aplicativos, como navegação, gerenciamento de estado, autenticação, mapeamento, gráficos e muito mais. Exemplos incluem React Navigation, Redux, Firebase, Axios, React Native Maps, entre outros.

Componentes prontos: além das bibliotecas, o ecossistema do React Native oferece uma variedade de componentes prontos para uso que as pessoas desenvolvedoras podem integrar em seus aplicativos para criar interfaces de usuário complexas e atraentes de maneira eficiente.

Comunidade ativa: a comunidade de desenvolvimento do React Native é extremamente ativa e vibrante, com milhares de desenvolvedores(as) contribuindo com códigos, compartilhando conhecimento e fornecendo suporte uns aos outros por meio de fóruns, grupos de discussão, conferências, meetups e redes sociais.

Documentação abundante: o React Native possui uma documentação abrangente e bem mantida que fornece informações detalhadas sobre como começar, guias de desenvolvimento, referências de API e tutoriais para ajudar os(as) profissionais de desenvolvimento a aprender e dominar a estrutura.

Ferramentas de teste: existem várias ferramentas de teste disponíveis para React Native que permitem a quem desenvolve testar seus aplicativos de forma eficaz, incluindo testes unitários, testes de integração e testes de interface do usuário.

Plataformas de distribuição: para distribuir aplicativos React Native, os(as) desenvolvedores(as) podem aproveitar plataformas como o Google Play Store e a

App Store da Apple. Além disso, existem serviços de hospedagem e distribuição de aplicativos que simplificam o processo de implantação e atualização de aplicativos.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Nesta aula sobre o ambiente de desenvolvimento mobile, exploramos o funcionamento geral desse cenário, incluindo as plataformas disponíveis e as abordagens para desenvolver aplicativos específicos ou multiplataformas. Analisamos as vantagens e desvantagens das tecnologias mais utilizadas nesse contexto, proporcionando uma visão abrangente das opções disponíveis para os desenvolvedores.

Além disso, destacamos a importância de configurar adequadamente o ambiente de desenvolvimento para garantir a fluidez e eficácia do processo de criação e execução de projetos. Essa configuração envolve desde a escolha das ferramentas até a preparação de ambientes de teste, assegurando um fluxo de trabalho tranquilo e produtivo para o desenvolvimento de aplicativos mobile.

REFERÊNCIAS

EXPO. **Documentação Oficial**. Expo. Disponível em: <https://docs.expo.dev>. Acesso em: 12 jul. 2024.

KOTLIN. **Documentação Oficial**. Kotlin. Disponível em: <https://kotlinlang.org/docs/getting-started.html>. Acesso em: 12 jul. 2024.

REACT NATIVE. **Documentação Oficial React Native**. Disponível em: <https://reactnative.dev>. Acesso em: 12 jul. 2024.

SWIFT. **Documentação OficialSwift**. Disponível em: <https://developer.apple.com/swift/>. Acesso em: 12 jul. 2024.

PALAVRAS-CHAVE

Palavras-chave: React Native. Expo. Mobile.

EMSE

POS TECH